Course: B.Tech CSE

Team ID: DSCPP-II-2026-T367

Title of the Project

Intelligent Digital Library and Book Recommendation System

PROJECT AND TEAM INFORMATION

Team Name: Team #

Team Member 1 (Team Lead)	Vanshika Goyal 2510376201 Vanshikagoyal701@gmail.com	
Team Member 2	Pragati Singhal 2510010644 pragatisinghal05@gmail.com	
Team Member 3	Prateek Bugana 2510375052 buganaprateek0274@gmail.com	

PROPOSAL DESCRIPTION

Motivation

Stand in front of a library shelf with a few thousand books on it and you'll see the problem pretty fast — there's no shortage of books, just no real way to figure out which one is worth picking up. Most college and community libraries are still running on old registers or a search box that only works if you type the exact title. Which is fine if you already know what you want. It's not much use if you don't. That's really the gap we're trying to close: a system that keeps track of the books properly on one side, and on the other, actually nudges a reader toward something new based on what they've read before. Nothing fancy — just enough to save people time, give smaller libraries a cheap way to modernize, and maybe make picking a book feel a bit more like getting a tip from a friend than digging through a dusty catalog.

State of the Art / Current Solution

Look at what's out there already and it's mostly the same handful of features: add a book, search by title or author, mark something issued or returned. Fine for record-keeping, but that's about where it stops — recommendations basically don't exist at this scale. Then you've got Goodreads and Amazon on the complete opposite end, doing recommendations really well, but they're massive commercial platforms, not something you can crack open and learn from as a student, let alone adapt for a small library. So that's where we're aiming: something in between, small enough to build and actually understand from the ground up, but still able to suggest books using nothing more exotic than the data structures and OOP concepts we're already learning in this course.

Project Goals and Milestones

In short, we want a digital library manager that stores, searches, issues, and returns books without any fuss, and that also recommends books based on what a user has already read. Here's roughly how we're planning to get there:

Milestone 1: Design the book and user data structures, then get the core operations — add, delete, search, issue, return — actually working in C.

Milestone 2: Speed things up with proper searching and sorting — linked lists, binary search trees, hashing, the works.

Milestone 3: Move over to C++ and design the object-oriented side of things for users, books, and the recommendation logic, using classes, inheritance, and polymorphism.

Milestone 4: Actually build the recommendation engine — matching on genre, author, and whatever the user has read before.

Milestone 5: Test it, fix what's broken (there will be something), and put together the final report and demo.

Project Approach

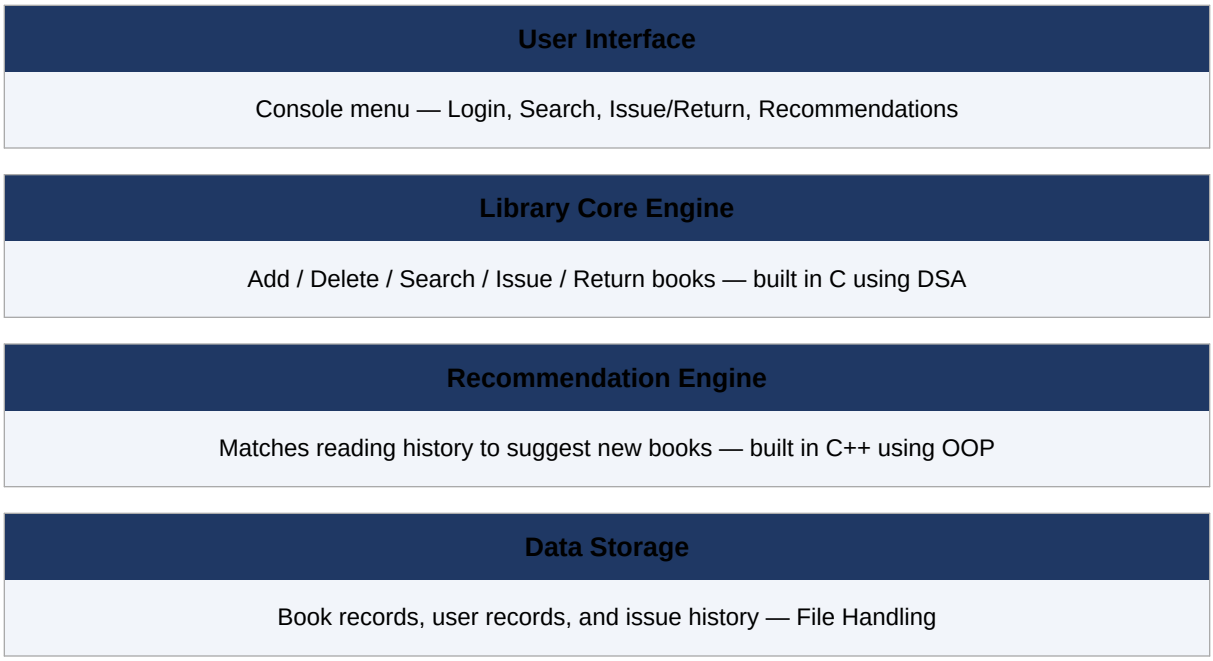
We're building this in two connected halves. The first is the core library engine, written in C, mostly because C gives us direct control over memory and speed — something that actually matters once the book list gets big. Books sit in a linked list, and we're using a binary search tree so we can look a title up

quickly instead of scanning through every single record like it's 1995. A hashing scheme takes care of fast lookups by a book's ID, and a couple of sorting routines let us arrange things by title, author, or popularity, whichever the situation calls for.

The second half is where the object-oriented programming comes in — the recommendation and user-management layer, written in C++. We're planning classes for Book, User, and Library, with inheritance handling the different account types (Student vs. Admin, mainly) and polymorphism so each type of account behaves the way it's supposed to without a mess of if-else checks everywhere. The recommendation part itself isn't as complicated as it sounds — compare what a user has already read against everything else in the catalog, and pull out the closest matches by genre or author. We'll build and test all of this on a regular machine with a standard C/C++ compiler like GCC, and we're keeping the interface a plain command-line menu on purpose, so the time goes into getting the logic right instead of fighting with a UI.

SYSTEM ARCHITECTURE (High Level)

Broken down, the system is really just four layers stacked on each other. Here's what that looks like:



Data moves top to bottom, basically: the user works through the console, the Library Core Engine handles the routine book operations, the Recommendation Engine digs through reading history to suggest new titles, and everything eventually lands in file-based storage.

Project Outcome / Deliverables

When it's done, we're handing in a working library application — add, search, issue, return, plus a recommendation feature that suggests titles to each user. Along with that: the full source code in C and C++, a sample dataset of books to test with, a short user guide, and a final report that walks through how we designed and tested the thing.

Assumptions

A couple of things we're taking for granted here. First, that the library's data — books and users — stays small enough for plain file storage to handle, no need for a full database server. Second, that only one person is using the system at a time, so we're not building for several people hitting it simultaneously in this version.

References

1. GeeksforGeeks — tutorials on data structures and algorithms (linked lists, BSTs, hashing, sorting).

2. GeeksforGeeks — object-oriented programming concepts in C++.

3. cplusplus.com — C++ Standard Library reference.

4. Course lecture notes and lab material.